

جامعة الجزائر 3 - كلية العلوم الاقتصادية
ماستر الاقتصاد الكمي - مادة :تطبيقات التعلم العميق

أعمال تطبيقية

حل سلسلة تمارين التعلم العميق:

الطالب: جرد محمد أمين

المدخلات

#

=====
=====

TP Deep Learning - Régression Linéaire avec TensorFlow/Keras
PROGRAMME PYTHON UNIQUE (CONFORME AUX CONSIGNES
DE L'ENSEIGNANT)

Auteur: Djerd Mohamed Amine

Environnement: Google Colab

#

=====
=====

TF/Keras هذا البرنامج يجب على جميع التمارين السبعة حول الانحدار الخطي باستخدام
ويتضمن: Normalisation, entraînement, MSE, extraction des poids,
diagnostic,

comparaison learning rate, correction du modèle, synthèse finale.

#

=====
=====

#

=====
=====

IMPORTATIONS

#

=====
=====

import numpy as np

import tensorflow as tf

from tensorflow import keras

```
from tensorflow.keras import layers, optimizers
```

```
#
```

```
=====
```

```
# تثبيت العشوائية لضمان نفس النتائج تقريباً عند كل تشغيل
```

```
#
```

```
=====
```

```
np.random.seed(42)
```

```
tf.random.set_seed(42)
```

```
#
```

```
=====
```

```
# PREPARATION DES DONNEES + NORMALISATION (MIN-MAX)
```

```
#
```

```
=====
```

```
print("="*90)
```

```
print("### PREPARATION DES DONNEES + NORMALISATION  
(MIN-MAX) ###")
```

```
print("="*90)
```

```
# بعلاقة خطية مثالية (Synthetic Data) بيانات تركيبية:
```

```
# Prix(k€) = 2 * Surface(m²)
```

```
X_raw = np.array([50, 70, 90, 110, 130, 150]).reshape(-1, 1)
```

```
y_raw = np.array([100, 140, 180, 220, 260, 300]).reshape(-1, 1)
```

```
print("Données originales:")
```

```
print("X_raw (surface m²) =", X_raw.flatten())
```

```
print("y_raw (prix k€)   =", y_raw.flatten())
```

```
# حساب Min و Max
```

```
x_min, x_max = X_raw.min(), X_raw.max()
```

```
y_min, y_max = y_raw.min(), y_raw.max()
```

```
print(f"\nBornes de X: min={x_min}, max={x_max}")
```

```
print(f"Bornes de y: min={y_min}, max={y_max}")
```

```

# دالة التطبيع Min-Max
def normalize_minmax(data, dmin, dmax):
    return (data - dmin) / (dmax - dmin)

# دالة إلغاء التطبيع
def denormalize_minmax(data_norm, dmin, dmax):
    return data_norm * (dmax - dmin) + dmin

# تطبيق التطبيع
X_norm = normalize_minmax(X_raw, x_min, x_max)
y_norm = normalize_minmax(y_raw, y_min, y_max)

print("\nDonnées normalisées:")
print("X_norm =", X_norm.flatten())
print("y_norm =", y_norm.flatten())

#
=====
=====
# EXERCICE 1 : VERIFICATION ENVIRONNEMENT
(TF/KERAS/GPU)
#
=====
=====
print("\n" + "="*90)
print("### EXERCICE 1 : VERIFICATION DE L'ENVIRONNEMENT
###")
print("="*90)

# Q1.1
print(f"Réponse 1.1: Version TensorFlow installée = {tf.__version__}")

# Q1.2
print(f"Réponse 1.2: Version Keras associée = {keras.__version__}")

# Q1.3
gpu_devices = tf.config.list_physical_devices('GPU')
is_gpu_available = len(gpu_devices) > 0

```

```
print(f"Réponse 1.3: GPU disponible ? => {'OUI' if is_gpu_available else 'NON'}")
print(" Méthode: tf.config.list_physical_devices('GPU')")
print(" Si la liste est vide => aucune GPU détectée.")
```

Q1.4

```
print("\nRéponse 1.4: Explication de l'absence de GPU et impact:")
```

```
explain_gpu = """
```

L'absence de GPU signifie que TensorFlow exécutera tous les calculs sur CPU.

Dans un modèle très simple comme une régression linéaire (une seule couche Dense),

le nombre d'opérations est faible et la différence de temps CPU/GPU est souvent négligeable.

Cependant, pour des réseaux profonds (CNN, RNN, Transformers) et des datasets volumineux,

l'absence de GPU augmente fortement le temps d'entraînement (minutes/heures au lieu de secondes).

```
"""
```

```
print(explain_gpu)
```

```
#
```

```
=====
```

```
# EXERCICE 2 : ENTRAINEMENT DU MODELE DE BASE (ADAM
```

```
lr=0.001, epochs=100)
```

```
#
```

```
=====
```

```
print("\n" + "="*90)
```

```
print("### EXERCICE 2 : ENTRAINEMENT DU MODELE DE BASE +
```

```
ANALYSE MSE ###")
```

```
print("="*90)
```

```
# بناء النموذج الأساسي (Linear Regression = Dense(1))
```

```
model_base = keras.Sequential([
```

```
    layers.Dense(1, input_shape=(1,), name="linear_output")
```

```
])
```

```
# Optimizer obligatoire حسب التعليم: Adam lr=0.001
optimizer_base = keras.optimizers.Adam(learning_rate=0.001)

# Compile
model_base.compile(optimizer=optimizer_base,
loss="mean_squared_error")

# Training obligatoire 100 epochs حسب التعليم:
history_base = model_base.fit(X_norm, y_norm, epochs=100, verbose=0)

# النهائي حساب MSE
mse_base = history_base.history["loss"][-1]

print(f"Résultat entraînement de base: MSE finale = {mse_base:.6f}")
```

```
# Q2.1
print("\nRéponse 2.1: Définition de la MSE:")
mse_definition = """
MSE (Mean Squared Error) = moyenne des carrés des erreurs entre y réel
et y prédit.
```

```
Formule mathématique:

$$MSE = (1/n) * \sum (y_i - \hat{y}_i)^2$$

où:
n = nombre d'échantillons
y_i = valeur réelle
 $\hat{y}_i$  = valeur prédite
"""
print(mse_definition)
```

```
# Q2.2
print("Réponse 2.2: Pourquoi une MSE élevée est anormale sur des
données normalisées [0,1] ?")
explain_mse_high = """
Dans notre cas, y_norm appartient à l'intervalle [0,1].
Un modèle bien entraîné doit donc produire des prédictions proches de
[0,1].
```

Si la MSE est élevée, cela signifie que les prédictions sont éloignées des valeurs réelles.

Cela peut se produire si le modèle ne converge pas correctement ou si les poids sont encore mal ajustés.

Ainsi, une valeur élevée indique généralement un apprentissage insuffisant ou instable.

```
"""
```

```
print(explain_mse_high)
```

```
# Q2.3
```

```
print("Réponse 2.3: Quelle valeur de MSE est acceptable ?")
```

```
good_mse = """
```

Pour une relation parfaitement linéaire et sans bruit comme ici,
un modèle entraîné correctement doit donner une MSE très proche de 0 (≈ 0.0000).

Toute valeur inférieure à 0.01 peut être considérée excellente dans ce contexte.

```
"""
```

```
print(good_mse)
```

```
# Q2.4
```

```
print("Réponse 2.4: Hypothèses expliquant une MSE élevée avant  
modification:")
```

```
hypotheses = """
```

Hypothèses possibles:

1) Nombre d'epochs insuffisant: le modèle n'a pas eu assez de temps pour apprendre.

2) Learning rate non optimal:

- trop petit => convergence lente

- trop grand => divergence ou oscillations

3) Initialisation aléatoire défavorable des poids.

```
"""
```

```
print(hypotheses)
```

```
#
```

```
=====
```

```
=====
```

EXERCICE 3 : EXTRACTION DES POIDS ET EQUATION DU MODELE

```

#
=====
=====
print("\n" + "="*90)
print("### EXERCICE 3 : EXTRACTION DE L'EQUATION DU
MODELE ###")
print("="*90)

# Q3.1 استخراج w و b
weights = model_base.get_weights()
w_norm = weights[0][0][0]
b_norm = weights[1][0]

print("Réponse 3.1: Extraction via model.get_weights():")
print(f"  w_norm = {w_norm:.6f}")
print(f"  b_norm = {b_norm:.6f}")

# Q3.2 المعادلة المعيارية
print("\nRéponse 3.2: Equation normalisée:")
print(f"  prix_norm = {w_norm:.6f} * surface_norm + {b_norm:.6f}")

# Q3.3 تحويل إلى الحجم الحقيقي (إلغاء التطبيع)
print("\nRéponse 3.3: Dé-normalisation pour retrouver l'équation réelle:")

x_range = x_max - x_min
y_range = y_max - y_min

# تحويل w و b
w_real = w_norm * (y_range / x_range)
b_real = (b_norm * y_range) + y_min - (w_real * x_min)

print("  On sait que:")
print(f"  surface_norm = (surface - {x_min}) / ({x_max} - {x_min})")
print(f"  prix_norm    = (prix - {y_min}) / ({y_max} - {y_min})")

print("\n  Après transformation, on obtient:")
print(f"  prix(k€) = {w_real:.6f} * surface(m²) + {b_real:.6f}")

# Q3.4 m² تحقق يدوي لمساحة 100
surface_test = 100

```

```
price_pred_manual = w_real * surface_test + b_real
```

```
print(f"\nRéponse 3.4: Vérification manuelle pour {surface_test} m²:")
print(f" Prix prédit manuellement = {price_pred_manual:.2f} k€")
print(" (Valeur théorique attendue: 200 k€ car prix = 2 * surface)")
```

```
#
```

```
=====
=====
```

```
# EXERCICE 4 : DIAGNOSTIC DE NON-CONVERGENCE +
COMPARAISON SYSTEMATIQUE LR
```

```
#
```

```
=====
=====
```

```
print("\n" + "="*90)
print("### EXERCICE 4 : DIAGNOSTIC NON-CONVERGENCE +
IMPACT LEARNING RATE ###")
print("="*90)
```

```
# Q4.1 (مثال توقعات سالبة)
```

```
print("Réponse 4.1: Analyse du problème des prédictions négatives:")
print(" Si le modèle produit des valeurs négatives alors que  $y_{\text{norm}} \in [0,1]$ ," )
print(" cela signifie que le modèle n'a pas convergé correctement.")
print(" Les poids w et b ne sont pas encore ajustés pour correspondre aux données.")
```

```
# Q4.2
```

```
print("\nRéponse 4.2: Paramètres responsables:")
```

```
params_resp = ""
```

```
Les deux paramètres principaux responsables d'un mauvais apprentissage
sont:
```

- 1) learning_rate (taux d'apprentissage)
- 2) epochs (nombre d'itérations d'entraînement)

```
""
```

```
print(params_resp)
```

```
# Q4.3
```

```
print("Réponse 4.3: Relation learning rate - vitesse de convergence:")
```

```
lr_relation = ""
```


- learning_rate trop petit: mise à jour faible des poids => convergence lente.
- learning_rate trop grand: mise à jour trop forte => oscillation/divergence.
Un learning_rate adapté permet une descente stable vers le minimum de la loss.

"""

```
print(lr_relation)
```

Q4.4

```
print("Réponse 4.4: Définition de la convergence et rôle de la loss:")
```

```
conv_def = """
```

La convergence signifie que la loss diminue puis se stabilise à une valeur faible.

En observant l'évolution de la loss au fil des epochs, on peut détecter si le modèle converge.

Si la loss reste grande ou augmente, le modèle n'est pas convergent.

```
"""
```

```
print(conv_def)
```

```
# -----
```

```
# (مطلوب في التعلیمة) learning rate مقارنة منهجية لآخر
```

```
# -----
```

```
print("\n--- Comparaison systématique de l'effet du learning_rate  
(obligatoire) ---")
```

```
learning_rates = [0.0001, 0.001, 0.1]
```

```
results = {}
```

```
for lr in learning_rates:
```

```
    test_model = keras.Sequential([  
        layers.Dense(1, input_shape=(1,))  
    ])
```

```
    test_optimizer = keras.optimizers.SGD(learning_rate=lr)
```

```
    test_model.compile(optimizer=test_optimizer, loss="mse")
```

```
    hist = test_model.fit(X_norm, y_norm, epochs=50, verbose=0)
```

```
    results[lr] = hist.history["loss"][-1]
```

```
for lr, loss_val in results.items():
```

```
    print(f" learning_rate={lr:<7} => Loss après 50 epochs =  
    {loss_val:.6f}")
```

```
print("""
Conclusion:
- LR très petit => loss reste élevée (apprentissage lent).
- LR moyen => meilleure convergence.
- LR grand => convergence rapide mais peut devenir instable si trop grand.
""")
```

```
#
=====
=====
```

```
# EXERCICE 5 : CORRECTION (lr=0.1, epochs=500, affichage loss
chaque 100 epochs)
```

```
#
=====
=====
```

```
print("\n" + "="*90)
print("### EXERCICE 5 : CORRECTION DU MODELE (lr=0.1,
epochs=500) ###")
print("="*90)
```

```
# modèle corrigé
model_corrected = keras.Sequential([
    layers.Dense(1, input_shape=(1,), name="linear_corrected")
])
```

```
optimizer_corrected = keras.optimizers.SGD(learning_rate=0.1)
model_corrected.compile(optimizer=optimizer_corrected,
loss="mean_squared_error")
```

```
# Callback pour afficher la loss chaque 100 epochs
class PrintLossEvery100(keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        if (epoch + 1) % 100 == 0:
            print(f" Epoch {epoch+1:03d} => Loss (MSE) =
{logs['loss']:.10f}")
```

```
print("Entraînement corrigé avec lr=0.1 et epochs=500:")
history_corrected = model_corrected.fit(
    X_norm, y_norm,
```

```

    epochs=500,
    verbose=0,
    callbacks=[PrintLossEvery100()]
)

```

```

final_loss_corrected = history_corrected.history["loss"][-1]

```

```

print("\nRéponse 5.4: Commentaire sur l'évolution de la loss:")

```

```

comment_loss = f"""

```

On remarque une baisse rapide de la loss au début, puis une baisse plus lente jusqu'à stabilisation.

La loss finale est très proche de 0 ({final_loss_corrected:.10f}), ce qui montre que le modèle a convergé.

```

"""

```

```

print(comment_loss)

```

```

#

```

```

=====
=====

```

EXERCICE 6 : INTERPRETATION DES RESULTATS FINAUX

```

#

```

```

=====
=====

```

```

print("\n" + "="*90)

```

```

print("### EXERCICE 6 : INTERPRETATION DES RESULTATS ###")

```

```

print("="*90)

```

```

# استخراج الوزن والانحياز النهائيين

```

```

w_corr, b_corr = model_corrected.get_weights()

```

```

w_corr_norm = w_corr[0][0]

```

```

b_corr_norm = b_corr[0]

```

```

w_corr_real = w_corr_norm * (y_range / x_range)

```

```

b_corr_real = (b_corr_norm * y_range) + y_min - (w_corr_real * x_min)

```

```

# Q6.1

```

```

print("Réponse 6.1: لماذا النتيجة متوقعة رياضياً؟")

```

```

exp1 = """

```

البيانات المنشأة هي بيانات تركيبية مثالية بدون ضوضاء وتحقق علاقة خطية تامة

```

prix = 2 * surface.

```

واحدة (أي انحدار خطي)، Dense بما أن النموذج المستخدم هو طبقة
فهو قادر نظرياً على تمثيل هذه العلاقة بشكل كامل
بعد التدريب الصحيح $b \approx 0$ و $w \approx 2$ لذلك من الطبيعي أن نجد

"""

```
print(exp1)
```

```
print(f"Equation finale apprise:")
```

```
print(f" prix(k€) = {w_corr_real:.4f} * surface(m²) + {b_corr_real:.4f}")
```

Q6.2

```
print("\nRéponse 6.2: دلالة  $MSE \approx 0$ ؟")
```

```
exp2 = """
```

تعني أن النموذج يطابق بيانات التدريب بشكل شبه كامل $MSE \approx 0$
في هذا المثال ذلك طبيعي لأن البيانات مثالية

لكن في التطبيقات الواقعية، البيانات تحتوي على ضوضاء

(حفظ البيانات بدلاً من تعميمها) Overfitting قد يكون ذلك مؤشراً على $MSE=0$ إذا أصبح

"""

```
print(exp2)
```

Q6.3

```
print("Réponse 6.3: حدود النموذج مع بيانات عقارية حقيقية:")
```

```
exp3 = """
```

حدود النموذج:

يفترض خطية العلاقة بين المساحة والسعر، بينما في الواقع العلاقة قد تكون غير خطية (1)

بينما السعر الحقيقي يعتمد على عدة عوامل (الموقع، (surface) يعتمد على متغير واحد فقط (2)
(...العمر، الغرف).

"""

```
print(exp3)
```

Q6.4

```
print("Réponse 6.4: تحسينان مقترحان:")
```

```
exp4 = """
```

تحسينات ممكنة:

...مثل الموقع، عدد الغرف، عمر البناء (Multivariate Regression) إدخال عدة متغيرات (1)

لالتقاط (Hidden Layers) استعمال نموذج غير خطي: شبكة عصبية متعددة الطبقات (2)

علاقات أعقد.

"""

```
print(exp4)
```

```

#
=====
=====
# EXERCICE 7 : RAPPORT SYNTHETIQUE (PAGE UNIQUE)
#
=====
=====
print("\n" + "="*90)
print("### EXERCICE 7 : SYNTHESE FINALE (RAPPORT) ###")
print("="*90)

report = ""
=====
=====
RAPPORT SYNTHETIQUE - Régression Linéaire avec
TensorFlow/Keras
=====
=====

```

1) Normalisation des données

Nous avons appliqué la normalisation Min-Max afin de ramener les valeurs dans [0,1].

Cette étape est importante car elle stabilise l'entraînement et améliore la convergence des algorithmes de descente de gradient.

2) Fonction de perte MSE

La MSE (Mean Squared Error) est la moyenne des carrés des erreurs entre y réel et y prédit.

Une MSE élevée indique un mauvais apprentissage, tandis qu'une MSE proche de 0 indique une bonne approximation des données.

3) Rôle des hyperparamètres (learning rate et epochs)

Le learning rate contrôle la vitesse de mise à jour des poids:

- trop petit => convergence lente
- trop grand => instabilité

Le nombre d'epochs représente le nombre de passages sur les données et influence

directement la qualité de convergence.

4) Extraction des paramètres du modèle

Nous avons extrait w et b via `model.get_weights()`, puis reconstruit l'équation normalisée:

$$\text{prix_norm} = w * \text{surface_norm} + b$$

Ensuite, nous avons effectué la dé-normalisation pour retrouver l'équation réelle en m^2 et k€ .

5) Evaluation des performances

La performance a été évaluée via la MSE. Après correction des hyperparamètres ($\text{lr}=0.1$, $\text{epochs}=500$), la loss est devenue très faible et le modèle a retrouvé la relation idéale: $\text{prix}(\text{k€}) \approx 2 * \text{surface}(\text{m}^2)$.

```
=====
=====
"""
print(report.strip())

print("\n" + "="*90)
print("FIN DU PROGRAMME - Toutes les questions des 7 exercices ont
été traitées.")
print("="*90)
```

المخرجات

```
=====
=====
### PREPARATION DES DONNEES + NORMALISATION (MIN-
MAX) ###
=====
=====
```

Données originales:

X_{raw} (surface m^2) = [50 70 90 110 130 150]

y_{raw} (prix k€) = [100 140 180 220 260 300]

Bornes de X: min=50, max=150

Bornes de y: min=100, max=300

Données normalisées:

```
X_norm = [0. 0.2 0.4 0.6 0.8 1. ]
y_norm = [0. 0.2 0.4 0.6 0.8 1. ]
```

```
#####
#####
### EXERCICE 1 : VERIFICATION DE L'ENVIRONNEMENT ###
#####
#####
```

Réponse 1.1: Version TensorFlow installée = 2.20.0

Réponse 1.2: Version Keras associée = 3.13.2

Réponse 1.3: GPU disponible ? => NON

Méthode: `tf.config.list_physical_devices('GPU')`

Si la liste est vide => aucune GPU détectée.

Réponse 1.4: Explication de l'absence de GPU et impact:

L'absence de GPU signifie que TensorFlow exécutera tous les calculs sur CPU.

Dans un modèle très simple comme une régression linéaire (une seule couche Dense),

le nombre d'opérations est faible et la différence de temps CPU/GPU est souvent négligeable.

Cependant, pour des réseaux profonds (CNN, RNN, Transformers) et des datasets volumineux,

l'absence de GPU augmente fortement le temps d'entraînement (minutes/heures au lieu de secondes).

```
#####
#####
### EXERCICE 2 : ENTRAINEMENT DU MODELE DE BASE + ANALYSE MSE ###
#####
#####
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a
layer. When using Sequential models, prefer using an `Input(shape)` object
as the first layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Résultat entraînement de base: MSE finale = 0.515021

Réponse 2.1: Définition de la MSE:

MSE (Mean Squared Error) = moyenne des carrés des erreurs entre y réel et y prédit.

Formule mathématique:

$$\text{MSE} = (1/n) * \sum (y_i - \hat{y}_i)^2$$

où:

n = nombre d'échantillons

y_i = valeur réelle

$\hat{y}_i$ = valeur prédite

Réponse 2.2: Pourquoi une MSE élevée est anormale sur des données normalisées [0,1] ?

Dans notre cas, y_{norm} appartient à l'intervalle [0,1].

Un modèle bien entraîné doit donc produire des prédictions proches de [0,1].

Si la MSE est élevée, cela signifie que les prédictions sont éloignées des valeurs réelles.

Cela peut se produire si le modèle ne converge pas correctement ou si les poids sont encore mal ajustés.

Ainsi, une valeur élevée indique généralement un apprentissage insuffisant ou instable.

Réponse 2.3: Quelle valeur de MSE est acceptable ?

Pour une relation parfaitement linéaire et sans bruit comme ici, un modèle entraîné correctement doit donner une MSE très proche de 0 (≈ 0.0000).

Toute valeur inférieure à 0.01 peut être considérée excellente dans ce contexte.

Réponse 2.4: Hypothèses expliquant une MSE élevée avant modification:

Hypothèses possibles:

- 1) Nombre d'epochs insuffisant: le modèle n'a pas eu assez de temps pour apprendre.
- 2) Learning rate non optimal:
 - trop petit => convergence lente
 - trop grand => divergence ou oscillations
- 3) Initialisation aléatoire défavorable des poids.

```
=====
##### EXERCICE 3 : EXTRACTION DE L'EQUATION DU MODELE #####
=====
```

Réponse 3.1: Extraction via `model.get_weights()`:

```
w_norm = -0.310771
b_norm = 0.096142
```

Réponse 3.2: Equation normalisée:

```
prix_norm = -0.310771 * surface_norm + 0.096142
```

Réponse 3.3: Dé-normalisation pour retrouver l'équation réelle:

On sait que:

```
surface_norm = (surface - 50) / (150 - 50)
prix_norm    = (prix - 100) / (300 - 100)
```

Après transformation, on obtient:

```
prix(k€) = -0.621541 * surface(m²) + 150.305516
```

Réponse 3.4: Vérification manuelle pour 100 m²:

Prix prédit manuellement = 88.15 k€

(Valeur théorique attendue: 200 k€ car $\text{prix} = 2 * \text{surface}$)

```
=====
##### EXERCICE 4 : DIAGNOSTIC NON-CONVERGENCE + IMPACT
LEARNING RATE #####
=====
```

Réponse 4.1: Analyse du problème des prédictions négatives:

Si le modèle produit des valeurs négatives alors que $y_{\text{norm}} \in [0,1]$,

cela signifie que le modèle n'a pas convergé correctement.
Les poids w et b ne sont pas encore ajustés pour correspondre aux données.

Réponse 4.2: Paramètres responsables:

Les deux paramètres principaux responsables d'un mauvais apprentissage sont:

- 1) `learning_rate` (taux d'apprentissage)
- 2) `epochs` (nombre d'itérations d'entraînement)

Réponse 4.3: Relation learning rate - vitesse de convergence:

- `learning_rate` trop petit: mise à jour faible des poids => convergence lente.
 - `learning_rate` trop grand: mise à jour trop forte => oscillation/divergence.
- Un `learning_rate` adapté permet une descente stable vers le minimum de la loss.

Réponse 4.4: Définition de la convergence et rôle de la loss:

La convergence signifie que la loss diminue puis se stabilise à une valeur faible.

En observant l'évolution de la loss au fil des `epochs`, on peut détecter si le modèle converge.

Si la loss reste grande ou augmente, le modèle n'est pas convergent.

--- Comparaison systématique de l'effet du `learning_rate` (obligatoire) ---

`learning_rate=0.0001` => Loss après 50 `epochs` = 0.082326

`learning_rate=0.001` => Loss après 50 `epochs` = 0.391564

`learning_rate=0.1` => Loss après 50 `epochs` = 0.061568

Conclusion:

- LR très petit => loss reste élevée (apprentissage lent).
- LR moyen => meilleure convergence.
- LR grand => convergence rapide mais peut devenir instable si trop grand.

=====
=====
EXERCICE 5 : CORRECTION DU MODELE (lr=0.1, epochs=500)
###
=====

Entraînement corrigé avec lr=0.1 et epochs=500:

Epoch 100 => Loss (MSE) = 0.0005434703
Epoch 200 => Loss (MSE) = 0.0000135239
Epoch 300 => Loss (MSE) = 0.0000003366
Epoch 400 => Loss (MSE) = 0.0000000084
Epoch 500 => Loss (MSE) = 0.0000000002

Réponse 5.4: Commentaire sur l'évolution de la loss:

On remarque une baisse rapide de la loss au début, puis une baisse plus lente jusqu'à stabilisation.

La loss finale est très proche de 0 (0.0000000002), ce qui montre que le modèle a convergé.

=====
=====
EXERCICE 6 : INTERPRETATION DES RESULTATS
=====

Réponse 6.1: لماذا النتيجة متوقعة رياضياً؟

البيانات المنشأة هي بيانات تركيبية مثالية بدون ضوضاء وتحقق علاقة خطية تامة
 $\text{prix} = 2 * \text{surface}$.

واحدة (أي انحدار خطي)، Dense بما أن النموذج المستخدم هو طبقة
فهو قادر نظرياً على تمثيل هذه العلاقة بشكل كامل
بعد التدريب الصحيح $b \approx 0$ و $w \approx 2$ لذلك من الطبيعي أن نجد

Equation finale apprise:

$$\text{prix}(\text{k€}) = 1.9999 * \text{surface}(\text{m}^2) + 0.0086$$

Réponse 6.2: وهل هي مرغوبة دائماً؟ $\text{MSE} \approx 0$ دلالة

تعني أن النموذج يطابق بيانات التدريب بشكل شبه كامل $MSE \approx 0$ في هذا المثال ذلك طبيعي لأن البيانات مثالية.

لكن في التطبيقات الواقعية، البيانات تحتوي على ضوضاء (حفظ البيانات بدلاً من تعميمها) Overfitting قد يكون ذلك مؤشراً على $MSE=0$ إذا أصبح.

حدود النموذج مع بيانات عقارية حقيقية: Réponse 6.3:

حدود النموذج:

- 1) يفترض خطية العلاقة بين المساحة والسعر، بينما في الواقع العلاقة قد تكون غير خطية
- 2) بينما السعر الحقيقي يعتمد على عدة عوامل (الموقع، (surface) يعتمد على متغير واحد فقط (... العمر، الغرف).

تحسينات مقترحة: Réponse 6.4:

تحسينات ممكنة:

- 1) ...مثل الموقع، عدد الغرف، عمر البناء (Multivariate Regression) إدخال عدة متغيرات
 - 2) لالتقاط (Hidden Layers) استعمال نموذج غير خطي: شبكة عصبية متعددة الطبقات
- علاقات أعقد.

EXERCICE 7 : SYNTHESE FINALE (RAPPORT)

RAPPORT SYNTHETIQUE - Régression Linéaire avec TensorFlow/Keras

1) Normalisation des données

Nous avons appliqué la normalisation Min-Max afin de ramener les valeurs dans $[0,1]$.

Cette étape est importante car elle stabilise l'entraînement et améliore la convergence

des algorithmes de descente de gradient.

2) Fonction de perte MSE

La MSE (Mean Squared Error) est la moyenne des carrés des erreurs entre y réel et y prédit.

Une MSE élevée indique un mauvais apprentissage, tandis qu'une MSE proche de 0 indique une bonne approximation des données.

3) Rôle des hyperparamètres (learning rate et epochs)

Le learning rate contrôle la vitesse de mise à jour des poids:

- trop petit => convergence lente
- trop grand => instabilité

Le nombre d'epochs représente le nombre de passages sur les données et influence directement la qualité de convergence.

4) Extraction des paramètres du modèle

Nous avons extrait w et b via `model.get_weights()`, puis reconstruit l'équation normalisée:

$$\text{prix_norm} = w * \text{surface_norm} + b$$

Ensuite, nous avons effectué la dé-normalisation pour retrouver l'équation réelle en m^2 et $k\text{€}$.

5) Evaluation des performances

La performance a été évaluée via la MSE. Après correction des hyperparamètres ($lr=0.1$, $epochs=500$), la loss est devenue très faible et le modèle a retrouvé la relation idéale: $\text{prix}(k\text{€}) \approx 2 * \text{surface}(m^2)$.

=====

=====

FIN DU PROGRAMME - Toutes les questions des 7 exercices ont été traitées.

=====